

GetPayin Laravel Project Documentation



Table of Contents

1. [Project Overview](#)
2. [Features](#)
3. [Installation Guide](#)
4. [Directory Structure](#)
5. [Key Components](#)
6. [Views](#)
7. [Contribution](#)
8. [License](#)
9. [Credits](#)

1. Project Overview

GetPayin is a comprehensive content management system built with Laravel that provides:

- User authentication and role management
- Post creation and scheduling
- Multi-platform content distribution
- Media management
- Analytics dashboard

Technology Stack:

- Laravel 10
- MySQL 5.7+
- Blade Templating
- Bootstrap 5 (optional)

2. Features

Core Functionality

- ▢ User authentication (Register/Login/Password Reset)
- ▢ Post management (CRUD operations)
- ▢ Platform integration (Facebook, Twitter, etc.)
- ▢ Image upload with validation
- ▢ Scheduled publishing

Technical Features

- ▢ RESTful API architecture
- ▢ Form Request validation
- ▢ Eloquent ORM relationships
- ▢ Queue system for scheduled posts
- ▢ Event-driven notifications

3. Installation

System Requirements

- PHP 8.2+
- Composer 2.0+
- Node.js 16+
- MySQL 5.7+

Setup Instructions

```
# Clone repository
git clone https://github.com/ShahendaHemdan/GetPayin.git
cd GetPayin

# Install dependencies
composer install
npm install

# Configure environment
cp .env.example .env
php artisan key:generate

# Set up database (edit .env first)
php artisan migrate --seed

# Compile assets
npm run dev

# Start development server
php artisan serve

# Start queue
php artisan queue:work

# Start scheduler
php artisan schedule:run
```

Environment Variables

Key variables to configure:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=

MAIL_MAILER=smtp
MAIL_HOST=mailhog
MAIL_PORT=1025
```

4. Directory Structure

```
app/  
├─ Http/  
│   └─ Controllers/  
│       └─ Blade/           # Web controllers  
│       └─ API/             # API controllers  
│       └─ Requests/        # Form validation  
config/                     # Configuration files  
database/  
├─ factories/               # Model factories  
├─ migrations/              # Database schemas  
└─ seeders/                 # Test data  
public/                     # Web root  
resources/  
├─ js/                      # JavaScript  
├─ sass/                    # Stylesheets  
└─ views/                   # Blade templates  
routes/                     # Web + API routes  
tests/                      # PHPUnit tests
```

5. Key Classes and Files

UpdatePostRequest

Location: `app/Http/Requests/Post/UpdatePostRequest.php`

```
public function rules()  
{  
    return [  
        'title' => 'sometimes|required|string|max:255',  
        'content' => 'sometimes|required|string',  
        'image' => 'sometimes|image|mimes:jpeg,png,jpg,gif,svg|max:2048',  
        'scheduled_at' => 'sometimes|nullable|date',  
        'platforms' => 'sometimes|required|array|min:1',  
        'platforms.*' => 'exists:platforms,id',  
        'status' => 'sometimes|required|in:draft,scheduled,published'  
    ];  
}
```

Controller Examples

PostController.php

```
public function store(StorePostRequest $request)  
{  
    $post = auth()->user()->posts()->create(  
        $request->validated()  
    );  
  
    if ($request->hasFile('image')) {
```

```

        $post->addMediaFromRequest('image')
            ->toMediaCollection('posts');
    }

    return redirect()->route('posts.show', $post);
}

```

6. Blade Views

Template Structure

```

views/
├─ layouts/
│   └─ app.blade.php      # Main layout
│   └─ guest.blade.php    # Auth pages layout
├─ auth/                  # Authentication views
├─ posts/                  # Post management
├─ components/            # Reusable components
└─ dashboard.blade.php    # Main dashboard

```

Example View: post.blade.php

```

@extends('layouts.app')

@section('content')
<div class="container">
    <h1>{{ $post->title }}</h1>

    @if($post->hasMedia('featured'))
    title }}" class="img-fluid">
    @endif

    <div class="content">
        {!! $post->content !!}
    </div>

    <div class="meta">
        Scheduled for: {{ $post->scheduled_at->format('M j, Y H:i') }}
    </div>
</div>
@endsection

```

7. Contribution

Workflow

1. Fork the repository
2. Create feature branch (`git checkout -b feature/your-feature`)
3. Commit changes (`git commit -am 'Add some feature'`)
4. Push to branch (`git push origin feature/your-feature`)

5. Create Pull Request

Coding Standards

- Follow PSR-12 coding style
- Write unit tests for new features
- Document complex logic with comments
- Keep commits atomic

8. License

MIT License - See [LICENSE](#) file for details.

9. Credits

Core Technologies

- [Laravel](#)
- [MySQL](#)
- [Bootstrap](#)

Dependencies

See `composer.json` and `package.json` for complete list

Documentation Version: 1.0

Last Updated: 27-5-2025 **Maintainer:** Shahenda shahindahemdan132@gmail.com